

A Comparative Software Benchmark of Lightweight Hash Functions on 8-bit AVR Using ChipWhisperer

Mohsin Khan¹[0000–0003–1815–8642]*, Dag Johansen¹[0000–0001–7067–6477], and
Håvard Dagenborg¹[0000–0002–1637–7262]

UiT: The Arctic University of Norway, Tromsø, Norway
{mohsin.khan, dag.johansen, havard.dagenborg}@uit.no

Abstract. Lightweight hash functions have become important building blocks for security in embedded and IoT systems. A plethora of algorithms have been proposed and standardized, providing developers with a wide range of performance trade-off options. This paper presents a comparative analysis of 22 software-based lightweight hash functions, including submissions from the NIST lightweight cryptography standardization process, ISO, and the SHA-3 competition. We use a novel benchmark methodology that combines an AVR ATXMega128 microcontroller with the ChipWhisperer power analysis toolset. We evaluate and compare the various hash functions along several dimensions, including execution speed, memory footprint, and energy consumption. Using the composite E-RANK metric, we provide insight into the various trade-offs and overall efficiency each hash function offers to system developers.

Keywords: Lightweight Cryptography · Hash Functions · Software Benchmarking · Performance Evaluation · NIST LWC · ISO/IEC · AVR · ChipWhisperer

1 Introduction

Hash functions are vital building blocks for many security mechanisms, like digital signatures, message authentication codes, and file checksums. Traditional hash functions are effective on devices with ample computational resources, but may fall short on resource-constrained devices with limited computational capacity, such as 8-bit AVR and PIC families of microcontrollers [35].

DM-PRESENT [17] was one of the first hash functions explicitly developed for devices with limited resources. From 2007 to 2012, NIST ran the SHA-3 competition to find a lighter and more secure alternative to SHA-2. Keccak [15] won this competition, with BLAKE as one of the finalists. These hash functions are used in cryptographic libraries today, such as OpenSSL [40] and Python’s hash-lib [42]. In 2016, ISO standardized PHOTON, SPONGENT [16], and Lesamnta-LW under ISO/IEC 29192-5:2016 [31]. In 2019, NIST began the standardization of lightweight cryptographic algorithms and received 57 submissions, some

* Corresponding author: Mohsin Khan, email: mohsin.khan@uit.no

of which included lightweight hash functions. After three rounds of evaluation and rigorous analysis of security and benchmarking results, ASCON [27] was announced as the winner in 2023. Consequently, many lightweight hashing functions have been proposed and developed, providing a wide range of performance and security trade-offs.

To the best of our knowledge, no existing publication provides a unified comparative analysis of lightweight hash functions spanning all standardization efforts, such as NIST LWC, ISO/IEC 29192-5, and the SHA-3 competition, evaluated on a single resource-constrained platform. A few studies have benchmarked lightweight AEAD ciphers on microcontrollers [25,43], but the lightweight hash functions are not mentioned. Some studies only examined a limited set of hash primitives under constrained power settings [45], and comprehensive surveys often lack hands-on software performance comparisons across various designs [49].

In this paper, we benchmark and compare 22 key lightweight hash functions on an 8-bit AVR platform, including those submitted to NIST [37], those standardized by ISO [31], including PHOTON and Lesamnta-LW, along with the SHA-3 competition [21] finalist, BLAKE [3] and its latest variant, BLAKE3 [39]. The selected hash functions are intended for deployment on highly resource-constrained devices, particularly those categorized by the IETF as Class 0 and Class 1 constrained nodes [18]. Class 0 devices depend on larger proxies or servers for Internet communication and typically possess $\ll$ 10 kB of RAM and $\ll$ 100 kB of flash storage. Class 1 devices can run simple applications but struggle to support full protocol stacks such as HTTP and TLS, usually featuring about 10 kB of RAM and 100 kB of flash storage. We target the AVR ATXMega128 microcontroller, a representative 8-bit platform widely used for its low power consumption, cost, and architectural simplicity. While 32-bit platforms, such as ARM Cortex-M, are increasingly popular, 8-bit microcontrollers remain prevalent in critical application domains, including industrial controllers and medical sensors. We focus exclusively on software performance metrics, including Cycles per Byte (CPB), energy consumption, RAM usage, and ROM using a novel benchmark environment comprised of the ATXMega128D4 in combination with the ChipWhisperer [38] power analysis tool. To quantify the trade-off between execution time, memory, and energy, we use the composite E-RANK metric [32].

2 ChipWhisperer

We run and observe the various hash functions using the ChipWhisperer Level 2 Starter Kit from NewAE Technology, a toolkit originally designed for learning about side-channel attacks on embedded devices by analyzing power consumption. Its modular design allows for the integration of specialized modules, enabling precise measurements and advanced testing of cryptographic systems. ChipWhisperer comprises a *capture board* and a *target board*, connected via a serial connection, and an oscilloscope probe, as shown in Figure 1.

The ChipWhisperer capture board includes an FPGA microcontroller (Xilinx Spartan-6), an Analog-to-Digital Converter (ADC), and a Low-Noise Amplifier

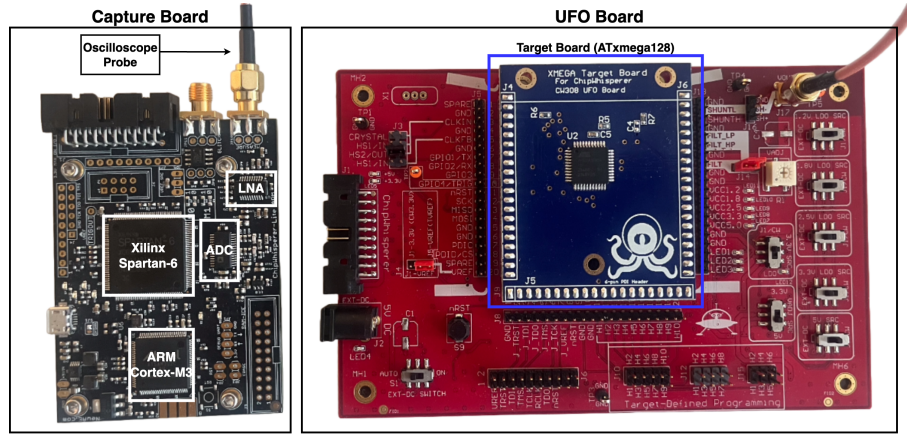


Fig. 1. ChipWhisperer hardware overview.

(LNA). The FPGA microcontroller serves as the central processing unit, managing communication, timing, and data acquisition processes. The LNA amplifies weak power signals to reduce noise and enhance the accuracy of the power analysis obtained from the target board. These amplified signals are then captured by the ADC for conversion into digital signals. The target board is mounted on a Universal Feature Observation (UFO) Board, which provides a standardized interface for power, clock, and data connections to the target board. Data is transferred between ChipWhisperer and the target board over a serial port, while the oscilloscope probe collects voltage traces as the target board executes specific processes triggered by this communication. The board also integrates an ARM Cortex-M3 as an on-board target, but this work uses only the external AVR ATXMEGA128D4 mounted on a UFO Board.

The ChipWhisperer Python API allows interaction with the ChipWhisperer FPGA, enabling programming of the target board, configuring clock settings, and triggering the target board to start and stop cryptographic operations. It also facilitates the capturing and transferring of power traces. Also, the Python API enables the configuration of ADC and LNA parameters, such as sampling rate and trigger settings, as well as adjustments to gain for capturing power traces and amplifying weak power signals.

The target board is programmed by uploading firmware, which is developed in the C language. This firmware includes a base C program designed to manage simple serial communication with the ChipWhisperer Python API. The base program also handles trigger signals from the Python API. The firmware is compiled using AVR-GCC, and C preprocessor macros are used to select the hash function (and its variant) to include in the object code. The implementations of the hash functions were sourced directly from their original publications. During the compilation process, the C implementations of the selected hash functions are

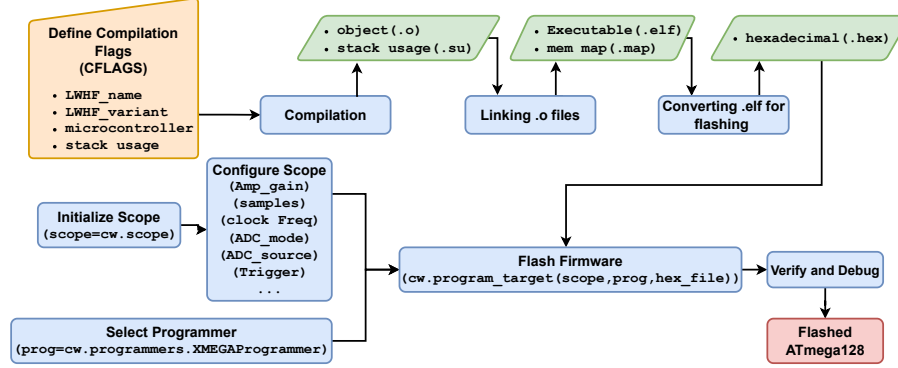


Fig. 2. Process flow for firmware build and deployment to target device.

converted into object files, and a corresponding `.su` file is generated to provide stack usage information. These object files are then linked together to create `.elf` and `.map` files. The `.elf` file is subsequently converted into a `.hex` file, which is used to flash the target device. The flashing is performed using the ChipWhisperer Python API, where the scope is initialized, and settings for amplifier gain, sampling rate, trigger conditions, and clock sources are configured. After flashing, the firmware is verified and debugged. The process for building firmware and deploying it on the target device using ChipWhisperer is illustrated in Figure 2.

3 Benchmarking Framework

We have opted for the AVR ATXMega128D4 microcontroller [2] as our target board for benchmarking the selected lightweight hash functions. It is a low-power 8-bit RISC microcontroller with 128 kB of flash memory, 8 kB of SRAM, 2 kB of EEPROM, a clock frequency of 32 MHz, and an operating voltage range of 1.6 to 3.6 V. These resource constraint specifications are sufficient to implement all the lightweight hash functions in this study and conduct performance tests. This setup also allows precise control over code execution and simplifies optimization at the assembly level.

Benchmarking begins by initializing the scope and configuring its settings. A 5 dB gain is applied to amplify low-amplitude power signals to minimize distortion, allowing clear capture of signal variations during hash function operations. A high-to-low trigger edge synchronizes the capture window with the start and the end of the hash function execution, and a sampling rate of 5 samples during the capture window offers moderate resolution for accurate trace alignment. The clock frequency is set to 7.3728 MHz using the ATXMega128D4’s internal clock. Simultaneously, a simple serial communication is established using UART. After setting up the simple serial interface and the scope, the target device is

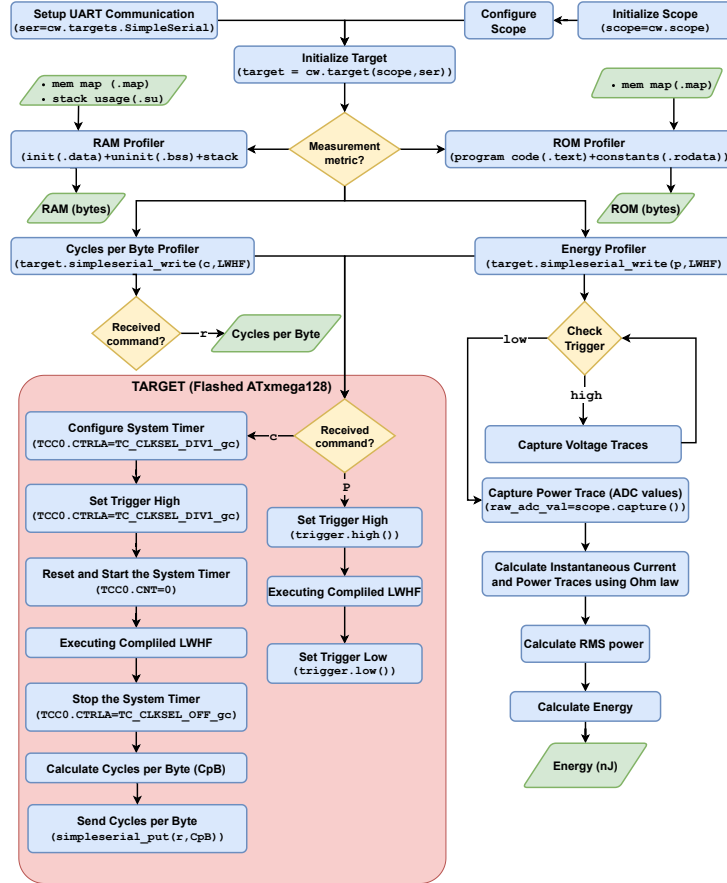


Fig. 3. Benchmarking framework.

initialized. Once the target initialization is complete, specific measurement metrics are selected for execution time, memory footprint, and energy profiling, as illustrated in the workflow of the benchmarking framework in Figure 3. We maintained the structure of each lightweight hash function by taking its original C language-based reference implementation to ensure consistency and fairness. No architecture-specific optimizations were implemented to ensure a fair comparison among all selected lightweight hash functions. The objective was to benchmark reference implementations using uniform compilation settings, rather than to achieve best-case optimized results for any algorithm. The full benchmarking framework, including source code and measurement scripts, is publicly available.¹

¹ https://github.com/khanmohsin/CryptoBench-ChipWhisperer-AVR_LWHF.git

3.1 Execution Time Profiling

We measure execution time in terms of the number of processing cycles required to hash each byte of data, commonly known as Cycles per Byte (CPB). Our CPB profiler utilizes a control register associated with the timer/counter on the ATXMega128D4 target board. According to the hardware specifications for the ATXMega128D4 microcontroller, the `TCC0.CTRLA` register is responsible for configuring the clock source and pre-scaler settings. When CPB measurement is selected, the `TCC0.CTRLA` register is initialized and set to zero at the beginning of the hashing process.

Once the hashing process has completed processing the input data, the control register is stopped, and the total number of cycles used is retrieved from the register. The resulting value is divided by the total number of input bytes processed to compute the final CPB value, before being transmitted to the host system. Note that the pre-scaler is set to `CLK_DIV1`, meaning that the timer/counter operates at the system clock frequency without any division.

3.2 Memory Footprint Profiling

In sponge constructions [14], the internal state is divided into two parts: rate and capacity. The rate indicates the number of bits of the state that can be directly read or written during the absorption and squeezing phases, determining the throughput of the function. The capacity is the portion of the state that remains hidden from input and output, providing security against cryptanalytic attacks. Merkle–Damgård constructions [24,34] do not explicitly separate the state into rate and capacity. Instead, the internal state is represented by the chaining value, while the rate is equal to the message block size. The overall state is updated by a compression function, and security is ensured through the design of that function and the padding scheme, rather than through a reserved hidden portion of the state.

For our purpose, we assess the memory footprint of each hash function by measuring its RAM and ROM consumption, which involves evaluating the size of the various memory segments used and summing up the total byte count. For RAM, we include the sizes of both initialized and uninitialized global and static variables, along with both dynamic and static stack usage, as shown below.

$$\text{RAM} = \text{.data} + \text{.bss} + \text{.su} \quad (1)$$

The initialized `.data` segment and the uninitialized `.bss` segment are extracted from the `.map` file that was generated by the AVR-GCC compiler during the linking stage. Both dynamic and static stack usage data are obtained from the `.su` files created during the compilation of individual object files, providing a comprehensive breakdown of memory allocation. Heap memory allocated via `malloc` and `calloc` calls are excluded because dynamic memory allocation is not utilized in the C implementations of the selected hash functions. For ROM, we sum the memory occupied by the program code `.text` segment and constant

data in the `.rodata` segment, as shown below.

$$\text{ROM} = \text{.text} + \text{.rodata} \quad (2)$$

The generated `.map` file provides a detailed memory map of the program code and constant data, allowing precise measurement of the total ROM footprint. The reported RAM and ROM values come from the compiled firmware image, which includes the benchmarking scaffold (SimpleSerial I/O and trigger handling) together with exactly one hash implementation selected at compile time. Since the scaffold overhead is constant across all builds, it does not affect the relative comparison between hash functions.

3.3 Energy Profiling

To profile energy consumption, we use the `cw.capture_trace` API function of the ChipWhisperer toolkit. The API call returns an array of instantaneous voltage samples collected by the ADC, normalized to the range -0.5 to +0.5. The voltage samples are measured when a high-to-low trigger signal is detected from the target device via serial communication.

While normalization ensures consistency across different hardware configurations, it requires us to convert the samples back to actual voltages. For this, let $\hat{V}$ be the normalized ADC voltage sample, V_{ref} the reference voltage of the ADC, which in our case is 1 V, and G the gain of the amplifier applied to the signal before digitization, which in our case is 5 dB, then the actual voltage V is given by

$$V = \frac{V_{\text{ref}}}{G} \times \hat{V} = 0.2\hat{V} \quad (3)$$

Furthermore, for the ATXMega128D4 microcontroller, we have the shunt resistance $R_{\text{shunt}} = 49.9 \Omega$ and the supply voltage $V_{\text{sup}} = 3.3 \text{ V}$. The instantaneous current I of a sample $\hat{V}$ is then given by Ohm's law as follows

$$I = \frac{V}{R_{\text{shunt}}} = \frac{V}{49.9} = \frac{0.2\hat{V}}{49.9} = 4.0 \times 10^{-3} \hat{V} \quad (4)$$

This gives us the instantaneous power P for sample $\hat{V}$ as follows.

$$P = I \times V_{\text{sup}} = 4.0 \times 10^{-3} \hat{V} \times 3.3 = 1.3 \times 10^{-2} \hat{V} \quad (5)$$

The ChipWhisperer's analog front end is AC-coupled, which causes the captured power traces to oscillate around zero, producing both positive and negative values. If one were to take a simple arithmetic mean of these samples, the result would underestimate actual power consumption due to signal cancellation. To avoid this distortion, we compute the Root Mean Square (RMS) power over the captured samples, which yields stable and comparable estimates of relative energy consumption across algorithms under AC-coupled conditions.

Given a trace $[\hat{V}_1, \hat{V}_2, \dots, \hat{V}_N]$ of N samples, the RMS power P_{rms} is given as follows

$$P_{\text{rms}} = \sqrt{\frac{1}{N} \sum_{i=1}^N P_i^2} = \sqrt{\frac{1.7 \times 10^{-4}}{N} \sum_{i=1}^N \hat{V}_i^2} \quad (6)$$

To calculate an accurate energy estimation, execution time needs to be considered, which represents the total duration the microcontroller spends executing the hash function. The execution time is determined using Equation 7, where T_{exec} denotes the execution period, C is the total number of processing cycles used during the hash function's execution, and f_{clk} indicates the microcontroller's clock frequency, which in our case is set to 7.3728 MHz. Once the execution period has been measured, the energy consumption can be calculated using Equation 8.

$$T_{\text{exec}} = \frac{C}{f_{\text{clk}}} = \frac{C}{7.3728 \text{ MHz}} \quad (7)$$

$$E = T_{\text{exec}} \times P_{\text{rms}} = \frac{C}{7.3728 \text{ MHz}} \times \sqrt{\frac{1.7 \times 10^{-4}}{N} \sum_{i=1}^N \hat{V}_i^2} \quad (8)$$

3.4 Performance Comparison and Ranking

While single-dimensional metrics, such as throughput, CPB, and energy consumption, are useful when comparing hash functions with similar optimization goals, they are not suitable for evaluating the tradeoff between various performance dimensions. For hardware implementations, Figure of Merit (FOM) [4] is commonly used as a compound metric that measures performance as the ratio of throughput to the square number of logical gates (i.e., GE). FOM also captures energy requirements, as power consumption is proportional to the number of logic gates, represented by the Gate Equivalent (GE) factor. For software implementations, the RANK metric [10] is commonly used to capture the trade-off between execution efficiency, expressed in CPB, and memory usage (RAM and ROM). However, unlike FOM, the included memory footprint of RANK does not offer any insights into power dissipation. To mitigate this limitation, we instead use the more recent E-RANK metric [32], as defined in Equation 9, which incorporates energy consumption E for a more detailed and accurate metric. E-RANK is a holistic benchmarking metric that combines execution speed, memory footprint, and energy consumption into a single measure. By integrating these dimensions, E-RANK highlights algorithms that achieve a balanced trade-off and penalizes those that are slow in execution or inefficient in terms of memory and energy consumption.

$$\text{E-RANK} = \frac{10^9/\text{cpb}}{(\text{ROM} + 2 \times \text{RAM}) \times E} \quad (9)$$

Table 1. Overview of selected lightweight hash function.

Name	Rate (bits)	Capacity (bits)	State (bits)	Structure	Internal Primitive	Parallelism	Rounds	Bit Affinity
ACE-H [1]	64	256	320	Sponge (sLiSCP-light)	ACE Permutation (Simeck-style)	None	48	32-bit
ASCON [27]	64	256	320	Sponge	Bit-sliced permutation	None	12/8	32-bit
BLAKE2s [3]	512	—	256	HAIFA (MD variant)	ChaCha-inspired G function	Tree	10	32-bit
BLAKE3 [39]	512	—	256	Binary Tree	BLAKE2s compression function	Tree (Merkle)	7	32-bit
CLX [50]	32	256	288	Sponge	$P'_{288,n}$ NLFSR permutation	None	var	8-bit
Coral [36]	32	224	256	Sponge	π permutation	None	10	64-bit
ESCH [11]	128	256	384	Modified Sponge	ARX-based Sparkle384	SIMD-friendly	var	32-bit
GAGE [28]	8	224	232	Sponge	Custom SPN permutation	None	32	8-bit
Gimli [13]	128	256	384	Sponge	Gimli permutation	None	24	32-bit
ISAP [26]	144	256	320 / 400	Sponge	Keccak-p[400] and Ascon-p	None	var	64-bit
KNOT [51]	32	224	256	Sponge/ Duplex	SPN-style substitution and diffusion	None	68	8-bit
Lesamnta-LW [30]	128	—	256	Merkle-Damgård (MD)	AES-based block cipher (LW1 mode)	None	64	8-bit
ORANGISH [20]	128	128	256	Sponge (JH mode)	PHOTON256 permutation	Intra-round (S-box/cols)	12	8-bit
PHOTON-256 [29]	32	256	288	Extended Sponge	AES-like permutation	Intra-round (S-box/cols)	12	8-bit
PHOTON-Beetle [7]	32	224	256	Sponge	PHOTON-256 permutation	Intra-round (S-box/cols)	12	8-bit
Saturnin [19]	256	—	256	Merkle Damgård	Saturnin Block Cipher	AES-like (bitsliced)	32	8-bit
SHAMAS [41]	64	256	320	Sponge/ Duplex	Bit-sliced permutation, linear matrix mixing, byte-wise rotations	Bit-sliced SIMD	12	64-bit
SIV-Rijndael [8]	32	224	256	Modified Sponge	Rijndael256 permutation	AES-like (bitsliced)	14	32-bit
SIV-TEM-PHOTON [9]	32	224	256	Modified Sponge	PHOTON-256 permutation	Intra-round (S-box/cols)	20	8-bit
SKINNY-tk2 [12]	32	224	256	Sponge	SKINNY-128-256 TK Cipher	None	48	8-bit
SNEIKHA [44]	256	256	512	Sponge (BLNK2)	SNEIK f512 ARX Permutation	None	8	32-bit
Subterranean [23]	32 (out) 9 (in)	224	257	Flat Sponge (Duplex)	Bitwise round function	None	1	32-bit
TRIAD [6]	32	224	256	Extended Sponge	Triad-P permutation	None	1024	8-bit
Xoodyak [22]	128	256	320	Duplex (Cyclist mode)	3×32-bit slices, XOR/rotate/shift	None	12	32-bit

4 Results and Analysis

An overview of all hash functions studied in his paper can be found in Table 1, including their internal parameters and structure. State refers to the total amount of internal memory allocated for use during the hash function’s operation.

The experimental results are visualized using graphs arranged in ascending order for execution time (CPB), memory footprint (both RAM and ROM), and energy consumption, while the composite E-RANK metric is presented in descending order. This arrangement places the most efficient hash function on the left. Performance values are analyzed in each results’ subsection across defined intervals or ranges, enabling a more detailed comparison. By segmenting the data this way, the analysis highlights which hash function performs best within each operational range.

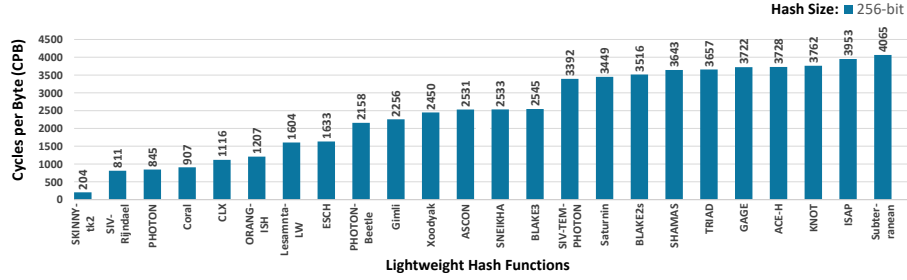


Fig. 4. Execution time of lightweight hash functions.

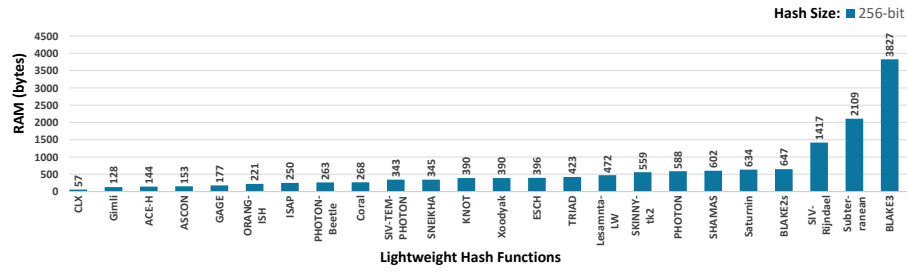


Fig. 5. RAM usage of the lightweight hash function.

4.1 Execution Time

The execution time is quantified using the Cycles per Byte (CPB) metric. A lower CPB value indicates higher computational efficiency and faster throughput, whereas a higher CPB value corresponds to slower processing and lower throughput. As shown in Figure 4, SKINNY-tk2 achieves the lowest CPB and is therefore the fastest candidate, followed closely by SIV-Rijndael and PHOTON-256. CLX, ORANGISH, and Lesamnta-LW comprises moderate efficiency. Algorithms such as ASCON, Gimli, and PHOTON-Beetle require more cycles, while ISAP and Subterranean are the slowest overall.

4.2 Memory Footprint

The memory footprint provides an estimate of the resource consumption of each hash function, evaluated in terms of RAM and ROM requirements.

RAM Consumption: As shown in Figure 5, several candidates such as CLX, Gimli, and ACE-H use only a few hundred bytes of RAM, placing them well within the budgets of Class 0 devices ($\ll 10$ kB RAM). Other designs, including PHOTON-256, SKINNY-tk2, and Lesamnta-LW, require larger but still modest RAM, remaining feasible for Class 0/Class 1 devices. At the higher end, BLAKE3 requires nearly 3.8 kB of RAM, which exceeds typical Class 0 limits but still fits within Class 1 constraints.

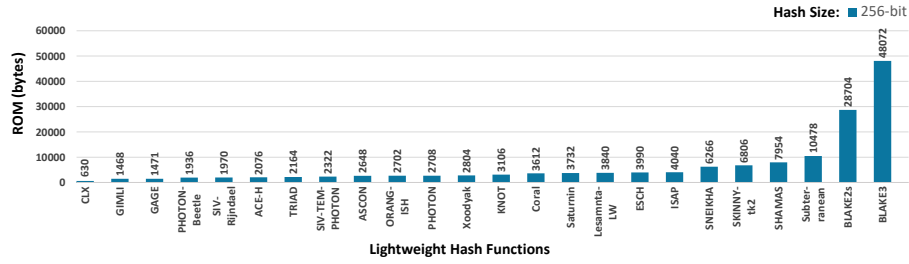


Fig. 6. ROM usage of the lightweight hash functions.

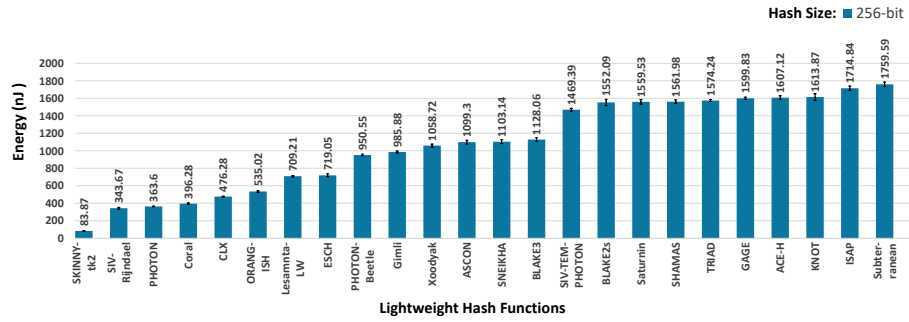


Fig. 7. Energy consumption of selected lightweight hash function.

ROM Consumption: Figure 6 illustrates that CLX and Gimli achieve the smallest code size, below the 100 kB flash limit of Class 0. Most candidates, including PHOTON-256, SKINNY-tk2, and Lesamnta-LW, also fall within Class 0 and Class 1 constraints. By contrast, BLAKE2s and BLAKE3 consume substantially more program memory than the other candidates. With about 48 kB of ROM, BLAKE3 remains within Class 1 limits but represents a significantly heavier footprint than designs like CLX or PHOTON.

Overall, three candidates (CLX, Gimli, ACE-H) fit Class 0; the majority, including PHOTON-256, SKINNY-tk2, and Lesamnta-LW fit within Class 0/Class 1, where the BLAKE family stands out as the most memory-intensive, though still within Class 1 constraints.

4.3 Energy Consumption

Figure 7 plots the measured energy consumption of the evaluated hash functions, arranged in ascending order. Each data point is the mean of 10 runs; error bars indicate variability across executions. The figure reflects the total energy required to process a fixed-length input and is particularly relevant for battery-powered or energy-constrained embedded systems.

The lowest energy consumption we observed is for SKINNY-tk2, followed by SIV-Rijndael. While both are among the most energy-efficient, SKINNY-tk2

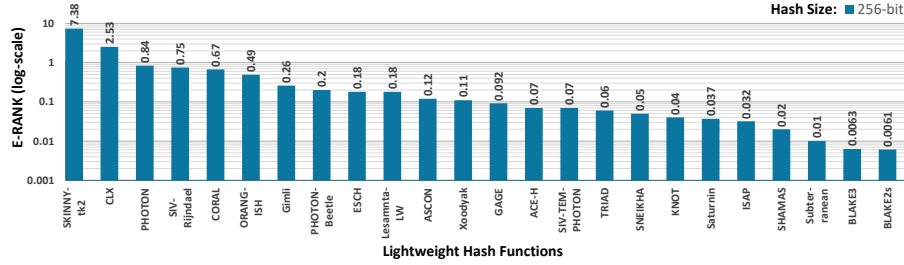


Fig. 8. E-RANK of the studied lightweight hash functions.

shows a notable margin of efficiency over SIV-Rijndael. CLX and Coral also achieve competitive efficiency, while PHOTON-256 provides balanced energy use. By contrast, Gimli, PHOTON-Beetle, and ASCON require noticeably more energy, and Subterranean, together with ISAP, are the most costly in terms of energy.

4.4 E-RANK

Figure 8 plots the E-RANK of the selected hash functions. From the figure, we observe that SKINNY-tk2 achieves the highest E-RANK, by a substantial margin, followed by CLX, which also attains an E-RANK greater than 1, demonstrating similarly strong efficiency characteristics. All remaining hash functions fall within the sub-unitary range ($\text{E-RANK} < 1$). Among these, PHOTON-256 exhibits the highest E-RANK, closely followed by SIV-Rijndael, both of which reflect favorable trade-offs under constrained-resource settings. Lesamnta-LW and ASCON achieve moderate E-RANK values, suggesting a reasonable balance, although not as optimized as the leading candidates. At the lower end of the spectrum, the BLAKE variants, specifically BLAKE2s and BLAKE3, record the lowest E-RANK values among all evaluated hash functions.

4.5 Comparative Analysis

Next, we present a comparative analysis of the benchmarked hash functions using a heatmap. For this, we first normalize memory, execution time, and energy consumption to a range between 0 and 1, using the min-max normalization method using Equation 10, where the value x is a single measurement and X the set of measurements.

$$\hat{x} = 1 - \frac{x - \min(X)}{\max(X) - \min(X)} \quad (10)$$

For E-RANK, values span several orders of magnitude with many hash functions clustered closely together around the lower end of the scale. To avoid having these values skewed and compressed by min-max normalization, making them

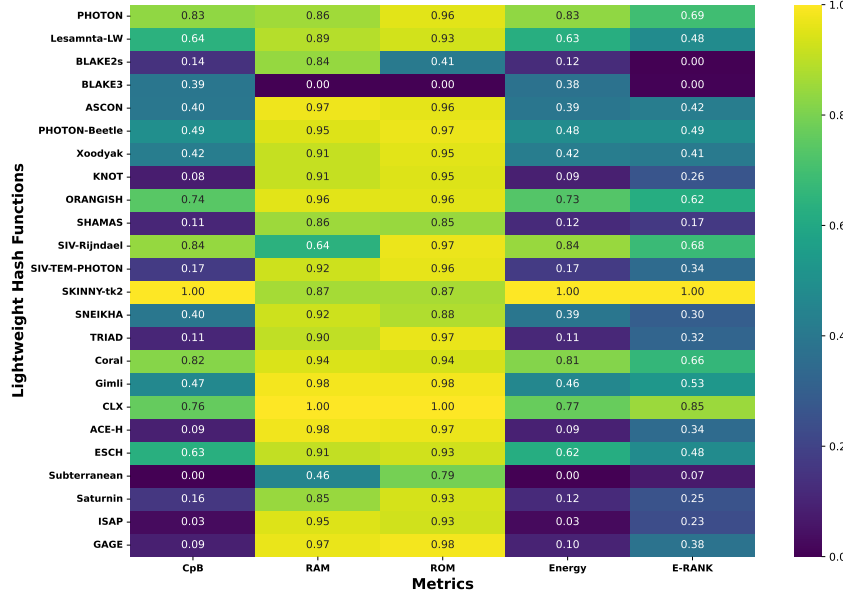


Fig. 9. Heatmap of selected lightweight hash functions (higher values are better).

hard to visualize in a heatmap, we apply a logarithmic transformation using the normalization Equation 11. The resulting heatmap is shown in Figure 9.

$$\bar{x} = \frac{\log(x \times \min(X))}{\log(\max(X)/\min(X))} \quad (11)$$

In terms of CPB and energy consumption, the evaluated hash functions SKINNY-tk2, SIV-Rijndael, PHOTON, and Coral exhibit the lowest CPB and energy usage. CPB is directly proportional to energy consumption. These hash functions utilize byte-aligned sponge constructions with fixed and non-branching permutations. They avoid runtime key scheduling, and the round functions consist of lightweight operations, such as 4-bit or 8-bit S-boxes, simple XORs, and linear diffusion layers. In contrast, ISAP, KNOT, and Subterranean have the lowest execution speeds and the highest energy consumption. This is primarily because these designs depend on bit-sliced or bit-level permutations, which consist of complex permutation structures. While these strategies enhance security and hardware optimization, they result in a higher CPB in software, illustrating the trade-off between security and efficiency on constrained devices.

In terms of memory footprint, CLX and Gimli have the least memory usage. These hash functions achieve compact memory footprints by relying on fixed-size, byte-aligned sponge states and avoiding large lookup tables or complex dynamic data structures. In contrast, the hash functions with the highest memory footprints, including BLAKE3, BLAKE2s, and Subterranean, exhibit structural

complexity. BLAKE3 and BLAKE2s utilize complex compression schedules involving numerous constants and key-dependent initialization values, along with tree-based or parallel chunk processing and extendable output. Meanwhile, Subterranean employs bitwise transformations and supports multiple operational wrappers, all of which contribute to high memory requirements.

Some hash functions, such as SKINNY-tk2, CLX, PHOTON, and SIV-Rijndael, achieve high E-RANK scores due to their low CPB, ROM, RAM, and energy consumption. Note that SKINNY-tk2, SIV-Rijndael, and PHOTON benefit specifically from low CPB and energy usage. In contrast, CLX does not have the fastest execution speed but has the lowest memory footprint among these functions.

Other hash functions, like BLAKE3, BLAKE2s, and Subterranean, show very low E-RANK scores due to their large memory footprints and high energy consumption. For example, the ROM and RAM requirements of BLAKE3 contribute to its lower E-RANK, while Subterranean suffers from poor CPB and energy efficiency combined with high RAM usage, which further reduces its E-RANK. These findings emphasize that overall balanced performance is not determined only by execution speed, but rather by a balanced integration of processing rate, memory, and power metrics. This also suggests that faster execution generally necessitates more memory.

Implementation Properties and Architecture Affinity. Table 1 lists each hash function with its bit affinity, determined through analysis of design specifications, including internal state structure, S-box granularity, and arithmetic operations. Algorithms optimized for 8-bit architectures (e.g., SKINNY-tk2, CLX, SIV-Rijndael) consistently exhibit better performance on the AVR ATXMega128 in terms of CPB, RAM usage, and energy consumption. In contrast, 32 and 64-bit oriented designs (e.g., BLAKE, ISAP, Subterranean) incur higher overhead due to memory misalignment. This analysis highlights how architectural alignment significantly influences real-world software performance on constrained devices.

5 Related Work

Few studies focus on benchmarking of software-based lightweight hash functions, other than the status reports published by NIST. There is a particular lack of comparative analysis with other standardized lightweight hash functions such as PHOTON, Lesamnta-LW, or BLAKE. To our knowledge, no other studies provide a quantitative evaluation or analytical assessment of the performance-to-cost tradeoff.

Balasch et al. [5] presented one of the earliest systematic benchmarks of hash functions on 8-bit AVR microcontrollers, targeting the ATtiny45 platform. They evaluated SHA-256, SHA-3 finalists, several lightweight designs such as PHOTON, SPONGENT, and Quark, as well as block-cipher-based constructions, focusing on code size, RAM usage, and cycle counts under strict implementation constraints. While highly valuable as a reference point, their study excluded ISO-

Table 2. Performance metrics calculated values of lightweight hash function.

Hash Function	CPB	RAM (B)	ROM (B)	Energy (nJ)	E-RANK
ACE-H	3728	144	2076	1607.12	0.07
ASCON	2531	153	2648	1099.3	0.12
BLAKE2s	3516	647	28704	1552.09	0.0061
BLAKE3	2545	3827	48072	1128.06	0.0063
CLX	1116	57	630	476.28	2.53
Coral	907	268	3612	396.28	0.67
ESCH	1633	396	3990	719.05	0.18
GAGE	3722	177	1471	1599.83	0.092
Gimli	2256	128	1468	985.88	0.26
ISAP	3953	250	4040	1714.84	0.032
KNOT	3762	390	3106	1613.87	0.04
Lesamnta-LW	1604	472	3840	709.21	0.18
ORANGISH	1207	221	2702	535.02	0.49
PHOTON-256	845	588	2708	363.6	0.84
PHOTON-Beetle	2158	263	1936	950.55	0.2
Saturnin	3449	634	3732	1559.53	0.037
SHAMAS	3643	602	7954	1561.98	0.02
SIV-Rijndael	811	1417	1970	343.67	0.75
SIV-TEM-PHOTON	3392	343	2322	1469.39	0.07
SKINNY-tk2	204	559	6806	83.87	7.38
SNEIKHA	2533	345	6266	1103.14	0.05
Subterranean	4065	2109	10478	1759.59	0.01
TRIAD	3657	423	2164	1574.24	0.06
Xoodyak	2450	390	2804	1058.72	0.11

standardized designs such as Lesamnta-LW, later NIST LWC submissions, and did not incorporate fine-grained energy profiling or composite efficiency metrics.

Khan et al. [33] performed benchmarking emphasizing hardware implementations. The study evaluates the throughput-to-area (TP/A) ratio, hardware area utilization, and execution time, but does not provide insights into the power analysis or software implementations. Windarta et al. [49] provide a detailed comparative study of NIST submitted hash functions, but the study concludes the results of multiple sources, such as previous research papers and internal evaluations by NIST. However, as a comparative study, it lacks a unified benchmarking approach to standardize performance comparisons.

The NIST status reports on lightweight cryptographic algorithms present software implementation results for NIST lightweight hash functions in detail, evaluating execution time, memory footprint, and power consumption on ARM Cortex-M4, ESP32, and AVR ATmega328P [46,47,48]. However, these benchmarking results lack fine-grained power profiling, tradeoff analysis between performance and cost, and performance tuning for specific embedded platforms.

6 Limitations and Future Work

The ChipWhisperer platform is primarily designed for side-channel analysis. However, we utilized this experimental study for high-resolution power and energy profiling. Future work should focus on conducting security analysis, such as side-channel assessments (e.g., differential power analysis), which would enhance performance benchmarking by providing insights into resilience.

7 Conclusions

This study presents a detailed methodology for conducting the first unified software benchmark that evaluates ISO-standardized lightweight hash functions, NIST-submitted candidates, and SHA-3 competition finalists from the BLAKE family on a single AVR platform, using precise ChipWhisperer-based power profiling. The experimental setup is designed to obtain precise measurements of key performance metrics. CPB are accurately captured by reading the on-chip hardware cycle counter. The memory footprint is derived from post-compilation analysis using the AVR-GCC toolchain. Energy consumption is measured using a power measurement probe connected to the ChipWhisperer’s integrated oscilloscope.

The results reveal clear trade-offs: faster execution often requires more memory (e.g., SKINNY-tk2), while designs emphasizing stronger security margins tend to execute more slowly (e.g., ISAP). ISO-standardized designs such as PHOTON and Lesamnta-LW, though optimized initially for hardware, prove highly suitable for software deployment on 8-bit platforms. Several NIST-LWC submissions, including SKINNY, CLX, and SIV-Rijndael, strike a strong balance across performance dimensions. By contrast, BLAKE2s and BLAKE3, while cryptographically robust and widely adopted in practice, impose significant memory and energy overheads on 8-bit AVR microcontrollers.

From a deployment perspective, these findings guide system designers. For Class 0 devices with limited memory and battery, algorithms like SKINNY-tk2, CLX, and PHOTON are ideal for tasks such as message authentication codes and lightweight digital signatures. Contrarily, BLAKE2s and BLAKE3 are better suited for Class 2 microcontrollers or server-side applications where memory and energy constraints are less critical. For file integrity checks, functions with stable throughput like Lesamnta-LW or ASCON are beneficial. Finally, the best hash function varies based on device class and application needs.

Disclosure of Interests. The authors have no conflicts of interest to declare.

References

1. Aagaard, M., AlTawy, R., Gong, G., Mandal, K., Rohit, R.: Ace: An authenticated encryption and hash algorithm. Submission document, NIST Lightweight Cryptography Project (2019), <https://csrc.nist.gov/CSRC/media/Projects/Lightweight-Cryptography/documents/round-1/spec-doc/ace-spec.pdf>

2. Atmel Corporation: ATxmega16D4/32D4/64D4/128D4 8/16-bit AVR Microcontroller Datasheet, https://ww1.microchip.com/downloads/en/DeviceDoc/Atmel-8135-8-and-16-bit-AVR-microcontroller-ATxmega16D4-32D4-64D4-128D4_datasheet.pdf
3. Aumasson, J., Meier, W., Phan, R.C.W., Henzen, L.: Blake2. In: The Hash Function BLAKE, pp. 165–183. Springer, Berlin, Heidelberg (2014). https://doi.org/10.1007/978-3-662-44757-4_9
4. Badel, S., Dağtekin, N., Nakahara, J., Ouafi, K., Reffé, N., Sepehrdad, P., Sušil, P., Vaudenay, S.: Armadillo: A multi-purpose cryptographic primitive dedicated to hardware. In: Mangard, S., Standaert, F.X. (eds.) Cryptographic Hardware and Embedded Systems – CHES 2010. Lecture Notes in Computer Science, vol. 6225, pp. 398–412. Springer, Berlin, Heidelberg (2010). https://doi.org/10.1007/978-3-642-15031-9_27
5. Balasch, J., Ege, B., Eisenbarth, T., Gérard, B., Gong, Z., Güneysu, T., Heyse, S., Kerckhof, S., Koeune, F., Plos, T., Pöppelmann, T., Regazzoni, F., Standaert, F., Van Assche, G., Van Keer, R., van Oldeneel tot Oldenzeel, L., von Maurich, I.: Compact implementation and performance evaluation of hash functions in attiny devices. In: Mangard, S. (ed.) Smart Card Research and Advanced Applications — CARDIS 2012. Lecture Notes in Computer Science, vol. 7771, pp. 158–172. Springer, Berlin, Heidelberg (2013). https://doi.org/10.1007/978-3-642-37288-9_11
6. Banik, S., Isobe, T., Meier, W., Todo, Y., Zhang, B.: Triad v1 – a lightweight aead and hash function based on stream cipher. Submission document, NIST Lightweight Cryptography Project (2019), <https://csrc.nist.gov/CSRC/media/Projects/Lightweight-Cryptography/documents/round-1/spec-doc/TRIAD-spec.pdf>
7. Bao, Z., Chakraborti, A., Datta, N., Guo, J., Nandi, M., Peyrin, T., Yasuda, K.: Photon-beetle: Authenticated encryption and hash family. Submission document, NIST Lightweight Cryptography Project (2021), <https://csrc.nist.gov/CSRC/media/Projects/Lightweight-Cryptography/documents/round-1/spec-doc/PHOTON-Beetle-spec.pdf>
8. Bao, Z., Guo, J., Iwata, T., Song, L.: Siv-rijndael256: Authenticated encryption and hash family. Tech. rep., NIST Lightweight Cryptography Project (Feb 2019), <https://csrc.nist.gov/CSRC/media/Projects/Lightweight-Cryptography/documents/round-1/spec-doc/SIV-Rijndael256-Spec.pdf>
9. Bao, Z., Guo, J., Iwata, T., Song, L.: Siv-tem-photon: Authenticated encryption and hash family. Tech. rep., NIST Lightweight Cryptography Project (Feb 2019), <https://csrc.nist.gov/CSRC/media/Projects/Lightweight-Cryptography/documents/round-1/spec-doc/SIV-TEM-PHOTON-Spec.pdf>
10. Beaulieu, R., Shors, D., Smith, J., Treatman-Clark, S., Weeks, B., Wingers, L.: The Simon and Speck block ciphers on AVR 8-bit microcontrollers. In: Eisenbarth, T., Öztürk, E. (eds.) Lightweight Cryptography for Security and Privacy. Lecture Notes in Computer Science, vol. 8898, pp. 3–20. Springer, Cham (2015). https://doi.org/10.1007/978-3-319-16363-5_1
11. Beierle, C., Biryukov, A., dos Santos, L.C., Großschädl, J., Perrin, L., Udovenko, A., Velichkov, V., Wang, Q.: Schwaemm and esch: Lightweight authenticated encryption and hashing using the sparkle permutation family. Submission document, NIST Lightweight Cryptography Project (2019), <https://csrc.nist.gov/CSRC/media/Projects/Lightweight-Cryptography/documents/round-1/spec-doc/SPARKLE-spec.pdf>

12. Beierle, C., Jean, J., Kölbl, S., Leander, G., Moradi, A., Peyrin, T., Sasaki, Y., Sasdrich, P., Sim, S.M.: Skinny-aead and skinny-hash: Authenticated encryption and hashing using the skinny block cipher. Tech. rep., NIST Lightweight Cryptography Project (2019), <https://csrc.nist.gov/CSRC/media/Projects/Lightweight-Cryptography/documents/round-1/spec-doc/SKINNY-spec.pdf>
13. Bernstein, D.J., Kölbl, S., Lucks, S., Massolino, P.M.C., Mendel, F., Nawaz, K., Schneider, T., Schwabe, P., Standaert, F.X., Todo, Y., Viguier, B.: Gimli: A cross-platform permutation for hashing and authenticated encryption. Submission document, NIST Lightweight Cryptography Project (2019), <https://csrc.nist.gov/CSRC/media/Projects/Lightweight-Cryptography/documents/round-1/spec-doc/gimli-spec.pdf>
14. Bertoni, G., Daemen, J., Peeters, M., Van Assche, G.: Sponge functions. In: ECRYPT Hash Workshop (2007), <https://keccak.team/files/SpongeFunctions.pdf>
15. Bertoni, G., Daemen, J., Peeters, M., Van Assche, G.: Keccak. In: Johansson, T., Nguyen, P.Q. (eds.) *Advances in Cryptology – EUROCRYPT 2013. Lecture Notes in Computer Science*, vol. 7881, pp. 313–314. Springer, Berlin, Heidelberg (2013). https://doi.org/10.1007/978-3-642-38348-9_19
16. Bogdanov, A., Knežević, M., Leander, G., Toz, D., Varici, K., Verbauwhede, I.: Spongent: A lightweight hash function. In: Preneel, B., Takagi, T. (eds.) *Cryptographic Hardware and Embedded Systems – CHES 2011. Lecture Notes in Computer Science*, vol. 6917, pp. 312–325. Springer, Berlin, Heidelberg (2011). https://doi.org/10.1007/978-3-642-23951-9_21
17. Bogdanov, A., Leander, G., Paar, C., Poschmann, A., Robshaw, M.J.B., Seurin, Y.: Hash functions and rfid tags: Mind the gap. In: Oswald, E., Rohatgi, P. (eds.) *Cryptographic Hardware and Embedded Systems – CHES 2008. Lecture Notes in Computer Science*, vol. 5154, pp. 283–299. Springer, Berlin, Heidelberg (2008). https://doi.org/10.1007/978-3-540-85053-3_18
18. Bormann, C., Ersue, M., Keränen, A.: Terminology for constrained-node networks. RFC 7228, Internet Engineering Task Force (IETF) (May 2014), <https://datatracker.ietf.org/doc/html/rfc7228>
19. Canteaut, A., Duval, S., Leurent, G., Naya-Plasencia, M., Perrin, L., Pornin, T., Schrottenloher, A.: Saturnin: A suite of lightweight symmetric algorithms for post-quantum security. Submission document, NIST Lightweight Cryptography Project (2019), <https://csrc.nist.gov/CSRC/media/Projects/Lightweight-Cryptography/documents/round-1/spec-doc/SATURNIN-spec.pdf>
20. Chakraborty, B., Nandi, M.: Orange: Algorithm specifications and supporting document. Tech. rep., NIST Lightweight Cryptography Project (2019), <https://csrc.nist.gov/CSRC/media/Projects/Lightweight-Cryptography/documents/round-1/spec-doc/orange-spec.pdf>
21. Chang, S.j., Perlner, R., Burr, W.E., Turan, M.S., Kelsey, J.M., Paul, S., Bassham, L.E.: Third-round report of the SHA-3 cryptographic hash algorithm competition. Tech. rep., NIST (2012), <https://nvlpubs.nist.gov/nistpubs/ir/2012/NIST.IR.7896.pdf>
22. Daemen, J., Hoffert, S., Peeters, M., Van Assche, G., Van Keer, R.: Xoodyak: A lightweight cryptographic scheme. *IACR Transactions on Symmetric Cryptology* pp. 60–87 (2020). <https://doi.org/10.13154/tosc.v2020.iS1.60-87>
23. Daemen, J., Massolino, P.M.C., Rotella, Y.: The subterranean 2.0 cipher suite. Submission document, NIST Lightweight Cryptography Project (2019), <https://csrc.nist.gov/CSRC/media/Projects/Lightweight-Cryptography/documents/round-1/spec-doc/subterranean-spec.pdf>

24. Damgård, I.B.: A design principle for hash functions. In: Brassard, G. (ed.) *Advances in Cryptology — CRYPTO’89 Proceedings*. pp. 416–427. Springer, New York, NY (1990). https://doi.org/10.1007/0-387-34805-0_39
25. Dinu, D., Biryukov, A., Großschädl, J., Khovratovich, D., Le Corre, Y., Perrin, L.: FELICS: Fair evaluation of lightweight cryptographic systems. In: *NIST Workshop on Lightweight Cryptography*. Gaithersburg, MD, USA (Jul 2015)
26. Dobraunig, C., Eichlseder, M., Mangard, S., Mendel, F., Mennink, B., Primas, R., Unterluggauer, T.: Isap v2.0: Submission to the nist lightweight cryptography standardization process. Submission document, NIST Lightweight Cryptography Project (2019), <https://csrc.nist.gov/CSRC/media/Projects/Lightweight-Cryptography/documents/round-1/spec-doc/ISAP-spec.pdf>
27. Dobraunig, C., Eichlseder, M., Mendel, F., Schläffer, M.: Ascon v1.2: Lightweight authenticated encryption and hashing. *Journal of Cryptology* **34**, 1–42 (2021). <https://doi.org/10.1007/s00145-021-09398-9>
28. Gligoroski, D., Mihajloska, H., Otte, D.: Gage and ingage v1.0: Submission to the nist lightweight cryptography standardization process. Submission document, NIST Lightweight Cryptography Project (2019), <https://csrc.nist.gov/CSRC/media/Projects/Lightweight-Cryptography/documents/round-1/spec-doc/GAGEandInGAGE-spec.pdf>
29. Guo, J., Peyrin, T., Poschmann, A.: The PHOTON family of lightweight hash functions. In: Rogaway, P. (ed.) *Advances in Cryptology – CRYPTO 2011. Lecture Notes in Computer Science*, vol. 6841, pp. 222–239. Springer, Berlin, Heidelberg (2011). https://doi.org/10.1007/978-3-642-22792-9_13
30. Hirose, S., Ideguchi, K., Kuwakado, H., Owada, T., Preneel, B., Yoshida, H.: A lightweight 256-bit hash function for hardware and low-end devices: Lesamnta-lw. In: Rhee, K.H., Nyang, D. (eds.) *Information Security and Cryptology – ICISC 2010. Lecture Notes in Computer Science*, vol. 6829, pp. 151–168. Springer, Berlin, Heidelberg (2011). https://doi.org/10.1007/978-3-642-24209-0_10
31. International Organization for Standardization: ISO/IEC 29192-5:2016 Information technology – Security techniques – Lightweight cryptography – Part 5: Hash-functions. Tech. rep., ISO/IEC (2016), <https://www.iso.org/standard/67173.html>, accessed: 30 Sept. 2025
32. Khan, M., Johansen, D., Dagenborg, H.: Performance evaluation of lightweight cryptographic ciphers on ARM processor for IoT deployments. In: Zhao, J., Meng, W. (eds.) *Science of Cyber Security. Lecture Notes in Computer Science*, vol. 17861, pp. 254–272. Springer Nature, Singapore (2025). https://doi.org/10.1007/978-981-96-2417-1_14
33. Khan, S., Lee, W.K., Karmakar, A., Mera, J.M.B., Majeed, A., Hwang, S.O.: Area-time efficient implementation of NIST lightweight hash functions targeting IoT applications. *IEEE Internet of Things Journal* **10**(9), 8083–8095 (2023). <https://doi.org/10.1109/JIOT.2022.3229516>
34. Merkle, R.C.: A certified digital signature. In: Brassard, G. (ed.) *Advances in Cryptology — CRYPTO’89 Proceedings*. pp. 218–238. Springer, New York, NY (1990). https://doi.org/10.1007/0-387-34805-0_21
35. Microchip Technology Inc.: 8-bit PIC and AVR Microcontrollers (2018), <https://ww1.microchip.com/downloads/en/DeviceDoc/30009630M.pdf>, accessed: 30 Sept. 2025
36. Montes, M., Penazzi, D.: Yarará and coral v1: Lightweight authenticated encryption and hash function algorithms. Submission document, NIST Lightweight Cryptography Project (2019), <https://csrc.nist.gov/CSRC/>

- media/Projects/Lightweight-Cryptography/documents/round-1/spec-doc/yarara_and_coral-spec.pdf
37. National Institute of Standards and Technology: Lightweight cryptography project (nd), <https://csrc.nist.gov/projects/lightweight-cryptography>, accessed: 22 Oct. 2024
 38. NewAE Technology Inc.: NewAE hardware product documentation (2025), <https://chipwhisperer.readthedocs.io/en/latest/>, accessed: 3 Mar. 2025
 39. O'Connor, J., Aumasson, J., Neves, S., Wilcox-O'Hearn, Z.: BLAKE3: One Function, Fast Everywhere. <https://github.com/BLAKE3-team/BLAKE3-specs/blob/master/blake3.pdf> (Jan 2020), accessed: 2025-03-24
 40. OpenSSL Project: Openssl cryptographic algorithms (2024), https://docs.openssl.org/3.4/man3/EVP_blake2b512/, accessed: 18 Aug. 2025
 41. Penazzi, D., Montes, M.: Shamash & shamashash: Lightweight authenticated encryption and hash function algorithms. Tech. rep., NIST Lightweight Cryptography Project (2019), <https://csrc.nist.gov/CSRC/media/Projects/Lightweight-Cryptography/documents/round-1/spec-doc/ShamashAndShamashash-spec.pdf>
 42. Python Software Foundation: hashlib — secure hashes and message digests (2024), <https://docs.python.org/3/library/hashlib.html>, accessed: 18 Aug. 2025
 43. Renner, S., Pozzobon, E., Mottok, J.: Benchmarking software implementations of first-round candidates of the NIST LWC project on microcontrollers. In: NIST Lightweight Cryptography Workshop. Gaithersburg, MD, USA (Aug 2019)
 44. Saarinen, M.J.O.: Sneiken and sneikha: Authenticated encryption and cryptographic hashing. Tech. rep., NIST Lightweight Cryptography Project (2019), <https://csrc.nist.gov/CSRC/media/Projects/Lightweight-Cryptography/documents/round-1/spec-doc/sneik-spec.pdf>
 45. Su, Y., Gao, Y., Kavehei, O., Ranasinghe, D.C.: Hash functions and benchmarks for resource constrained passive devices: A preliminary study. In: 2019 IEEE International Conference on Pervasive Computing and Communications Workshops (PerCom Workshops). pp. 1020–1025. IEEE (2019). <https://doi.org/10.1109/PERCOMW.2019.8730835>
 46. Turan, M.S., McKay, K., Chang, D., Bassham, L.E., Kang, J., Waller, N.D., Kelsey, J.M., Hong, D.: Status report on the final round of the NIST lightweight cryptography standardization process. Tech. rep., NIST (2023), <https://nvlpubs.nist.gov/nistpubs/ir/2023/NIST.IR.8454.pdf>
 47. Turan, M.S., McKay, K., Chang, D., Calik, C., Bassham, L., Kang, J., Kelsey, J.: Status report on the second round of the NIST lightweight cryptography standardization process. Nist internal report, NIST (2021)
 48. Turan, M.S., McKay, K.A., Çalik, Ç., Chang, D., Bassham, L.E., Kelsey, J., et al.: Status report on the first round of the NIST lightweight cryptography standardization process. NIST Internal Report NISTIR 8214, NIST (2019)
 49. Windarta, S., Suryadi, S., Ramli, K., Pranggono, B., Gunawan, T.S.: Lightweight cryptographic hash functions: Design trends, comparative study, and future directions. IEEE Access **10**, 82272–82294 (2022). <https://doi.org/10.1109/ACCESS.2022.3195572>
 50. Wu, H., Huang, T.: Clx: A family of lightweight authenticated encryption algorithms. Submission document, NIST Lightweight Cryptography Project (2019)
 51. Zhang, W., Ding, T., Yang, B., Bao, Z., Xiang, Z., Ji, F., Zhao, X.: Knot: Authenticated encryption and hash function family. Tech. rep., NIST Lightweight Cryptography Project (2019), <https://csrc.nist.gov/CSRC/media/Projects/Lightweight-Cryptography/documents/round-1/spec-doc/KNOT-spec.pdf>